🏗️

Reusable Enterprise AWS Terraform Modules — Project Documentation

Project overview

Project: Reusable Enterprise AWS Terraform Modules
Category: Infrastructure as Code (IaC) / AWS / Terraform
Purpose: Build reusable Terraform modules for common AWS infrastructure and parameterize them for development, staging, and production environments.

This document is a technical reference, deployment guide, and portfolio handover. It distinguishes configuration and planning completed during the build from infrastructure deployment and operational testing that have not been demonstrated.

> 🎯 Project objective: standardize repeatable AWS infrastructure through reusable Terraform modules, typed inputs, outputs, validation, resource tags, and environment-specific variable files.

1. Scope and delivery status

In scope

- Terraform root configuration and AWS provider setup.
- Reusable VPC module with VPC, public/private subnets, internet gateway, route tables, and subnet associations.
- Reusable security-group module with configurable ingress and egress rules.

- Reusable EC2 module using an Amazon Linux 2023 AMI parameter from AWS Systems Manager Parameter Store.
- Reusable CloudWatch CPU alarm module.
- DEV, STAGING, and PROD variable files.
- Formatting, validation, and plan workflows.
- Project documentation and GitHub handover material.

Explicitly out of scope for this iteration

- S3 remote Terraform backend.
- S3 state versioning and native S3 lockfiles.
- Remote-state migration and independent remote state keys.
- Full CI/CD automation.
- A production deployment or production-readiness certification.

The S3 backend was discussed as a possible future enhancement, but the project decision is to document it only and not implement it now. The project should be treated as using local state unless the repository shows otherwise.

Status summary

Area	Status based on project conversation
Project folder structure	Created
AWS provider configuration	Configured; credentials were troubleshot
VPC module	Implemented
Security group module	Implemented
EC2 module	Implemented
CloudWatch monitoring module	Implemented
DEV variable file	Created; plan reported successful
STAGING variable file	Created; plan reported successful
PROD variable file	Created; plan reported successful
S3 remote backend	Out of scope / not implemented

Area	Status based on project conversation
Actual AWS apply and runtime verification	Not confirmed
Full file-by-file code audit	Not completed; source archive was not supplied
GitHub README and diagram	Documentation deliverable in progress

A successful `terraform plan` means Terraform generated a proposed change set for the supplied configuration and credentials. It does not prove resources were created, connectivity works, or the deployment is production-ready.

2. Architecture

Logical architecture

The root Terraform configuration composes four local modules:

1. **VPC** — creates a VPC, public and private subnets, an internet gateway, route tables, and subnet associations.
2. **Security Group** — creates a security group and configurable ingress/egress rules.
3. **EC2** — reads an Amazon Linux 2023 AMI parameter and creates an EC2 instance in a supplied subnet with supplied security groups.
4. **Monitoring** — creates a CloudWatch CPU utilization alarm for the EC2 instance.

Resource dependency flow

```
Root Terraform configuration
        |
        +--> VPC module
        |      +--> VPC
        |      +--> Public subnets
        |      +--> Private subnets
        |      +--> Internet gateway
        |      +--> Public/private route tables
        |      +--> Route table associations
        |
        +--> Security Group module
```

```
        |      +--> Security group
        |      +--> Ingress rules
        |      +--> Egress rules
        |
        +--> EC2 module
        |      +--> SSM Parameter Store AMI lookup
        |      +--> EC2 instance
        |      +--> Encrypted gp3 root volume
        |
        +--> Monitoring module
               +--> CloudWatch CPU alarm
```

Module data flow

- The root module passes VPC inputs to the VPC module.
- The VPC module exports `vpc_id`, `vpc_cidr`, `public_subnet_ids`, and `private_subnet_ids`.
- The root module passes `module.vpc.vpc_id` to the security group module.
- The root module passes `module.vpc.public_subnet_ids[0]` and the security-group ID to the EC2 module.
- The EC2 module exports its instance ID and IP addresses.
- The root module passes `module.ec2.instance_id` to the monitoring module.

This establishes an explicit dependency chain through module outputs and inputs.

3. Repository structure

The structure created during implementation:

```
reusable-aws-terraform/
├── environments/
│   ├── dev/
│   │   └── terraform.tfvars
│   ├── staging/
│   │   └── terraform.tfvars
│   └── prod/
│       └── terraform.tfvars
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
```

```
│   ├── ec2/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── security-group/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── monitoring/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── main.tf
├── providers.tf
├── variables.tf
├── outputs.tf
├── .gitignore
└── docs/
    └── terraform-state-management.md
```

File responsibilities

File	Responsibility
`providers.tf`	Terraform version constraint, AWS provider source/version, provider region
Root `main.tf`	Calls modules and wires inputs/outputs
Root `variables.tf`	Declares environment and infrastructure inputs
Root `outputs.tf`	Exposes selected module outputs
Environment `terraform.tfvars`	Provides environment values
Module `main.tf`	Declares resources and data sources for the module
Module `variables.tf`	Defines the module input contract
Module `outputs.tf`	Exposes values needed by parent modules or operators
`.gitignore`	Excludes local state, provider cache, plans, and sensitive local files
`docs/`	Stores architecture, operations, and project decisions

4. Terraform and AWS provider configuration

The root provider configuration discussed uses Terraform minimum version 1.5.0 and the HashiCorp AWS provider constraint `~> 6.0`.

Representative configuration:

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

The target region is `ap-south-1` (Mumbai). A Region and an Availability Zone are different:

- Region: `ap-south-1`
- Availability Zones: account-visible zones in that region, for example `ap-south-1a` and `ap-south-1b`.

Availability Zone letter mappings can differ between AWS accounts. Check which zones are available to the account before deployment.

Credential handling

Terraform's AWS provider can use the standard AWS credential provider chain, including environment variables, shared AWS CLI profiles, and supported role/SSO credentials. Do not put access keys, secret keys, or session tokens in Terraform files or variable files.

Read-only checks:

```
aws --version
aws configure list
aws sts get-caller-identity
```

The project encountered STS `InvalidClientTokenId` during development. This indicates an invalid, expired, or mismatched credential source, not a VPC-module syntax issue. Authentication was subsequently treated as resolved in the project workflow. Never record credential values in project documentation.

5. VPC module

Path: `modules/vpc/`

Purpose

Provide a reusable network foundation with one VPC, multiple public subnets, multiple private subnets, and basic route-table associations.

Inputs

Variable	Type	Purpose
`vpc_name`	string	Name prefix for network resources
`vpc_cidr`	string	VPC IPv4 CIDR
`public_subnet_cidrs`	list(string)	Public subnet CIDR list
`private_subnet_cidrs`	list(string)	Private subnet CIDR list
`availability_zones`	list(string)	AZ values corresponding to subnet indexes
`tags`	map(string)	Common resource tags

The VPC CIDR variable has a validation check using `cidrnetmask` to reject invalid IPv4 CIDR notation.

Resources

- `aws_vpc.this`: creates the VPC and enables DNS support and DNS hostnames.
- `aws_subnet.public`: creates public subnet resources using `count`.
- `aws_subnet.private`: creates private subnet resources using `count`.

- `aws_internet_gateway.this`: attaches an internet gateway to the VPC.
- `aws_route_table.public`: provides a default route to the internet gateway.
- `aws_route_table.private`: creates a private route table without a default internet route.
- `aws_route_table_association.public`: associates public subnets to the public route table.
- `aws_route_table_association.private`: associates private subnets to the private route table.

Outputs

- `vpc_id`
- `vpc_cidr`
- `public_subnet_ids`
- `private_subnet_ids`

Networking behaviour and limitations

A subnet is not internet-reachable merely because its name or tag says “public”. The public route table's default route to an attached internet gateway is a necessary routing component. An instance also needs a public IPv4 address (or another appropriate addressing path) and security-group/NACL permissions for the intended traffic.

Private subnets currently have no NAT gateway and no default route to the internet. This is a cost-conscious baseline, but private instances will not have general outbound internet connectivity through this design.

The module uses indexes to pair subnet CIDRs and Availability Zones. Validate that list lengths match. A robust module should add validation or preconditions requiring:

- `length(public_subnet_cidrs) <= length(availability_zones)`
- `length(private_subnet_cidrs) <= length(availability_zones)`
- CIDR blocks are valid and do not overlap.

- Subnet CIDRs are contained within the VPC CIDR.

The project conversation does not establish that these additional validations were implemented.

6. Terraform language concepts used

Input variables

Variables create a module interface. The parent/root configuration passes values to child modules. The child module must declare each accepted input in its own `variables.tf`.

Data sources versus resources

- A Terraform `resource` manages an object, usually creating or updating it.
- A Terraform `data` block reads information about an existing object or published value.

The `count` meta-argument

`count` creates a number of instances of a resource or module. Indexing begins at zero.

```
resource "aws_route_table_association" "public" {
  count = length(aws_subnet.public)

  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}
```

If two public subnets exist, count is two and the index values are 0 and 1. Each generated association connects one subnet to the public route table.

The `merge()` function

`merge()` combines maps. Later maps override earlier maps when the same key occurs.

```
tags = merge(
  var.tags,
  {
    Name = var.vpc_name
  }
)
```

This combines common tags (Environment, Project, ManagedBy) with a resource-specific Name tag. Keep the resource-specific map later when it must override a shared key.

Outputs

Outputs expose useful values from child modules, allowing parent modules to connect infrastructure without hardcoding resource identifiers.

7. Security group module

Path: `modules/security-group/`

Purpose

Create a VPC security group with caller-supplied inbound and outbound rule definitions.

Inputs

- `security_group_name` — security group name.
- `description` — purpose/description.
- `vpc_id` — VPC in which the group is created.
- `ingress_rules` — list of objects containing description, from_port, to_port, protocol, and cidr_blocks.
- `egress_rules` — list of objects with the same fields.
- `tags` — common tags.

Resources

- `aws_security_group.this`
- `aws_vpc_security_group_ingress_rule.this`
- `aws_vpc_security_group_egress_rule.this`

The ingress and egress resources use `count` over the configured rule lists.

Outputs

- `security_group_id`
- `security_group_name`

Example rules in the root configuration

The web security group configuration discussed permits:

- TCP 80 from `0.0.0.0/0`
- TCP 443 from `0.0.0.0/0`
- TCP 443 outbound to `0.0.0.0/0`

SSH is not opened to the internet in this example.

Important implementation caveat

The initial implementation accepts `cidr_blocks` as a list but sets `cidr_ipv4` only when the list has exactly one element; otherwise it sets that property to null. This means the current resource code does not fully implement arbitrary multiple IPv4 CIDRs per rule. Before using multiple CIDRs, revise the module to use one rule resource per CIDR (for example, a `for_each` over flattened rule/CIDR pairs), or model a single CIDR per rule in the input type. Validate IPv6 and prefix-list behaviour separately if needed.

The module should also have explicit, reviewed egress semantics. The example config includes a restricted HTTPS egress rule; verify the provider's security-group defaults and the final planned rule set rather than assuming all outbound traffic is allowed or denied.

Public HTTP/HTTPS ingress may be appropriate for a public web endpoint, but must be deliberately approved. For administration, prefer AWS Systems Manager Session Manager or a tightly restricted management path rather than public SSH.

8. EC2 module

Path: `modules/ec2/`

Purpose

Create a configurable EC2 instance in a supplied subnet and attach supplied security groups.

Inputs

Variable	Type	Purpose
`instance_name`	string	Name tag
`instance_type`	string	EC2 instance type
`subnet_id`	string	Target subnet
`security_group_ids`	list(string)	Security groups to attach
`root_volume_size`	number	Root EBS size in GiB
`tags`	map(string)	Common tags

The root volume input has a minimum-size validation of 8 GiB.

AMI lookup using SSM Parameter Store

The EC2 module reads this public Systems Manager parameter:

```
data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-ker
nel-default-x86_64"
}
```

This data source reads the current parameter value in the selected Region. The returned `value` is the AMI ID used by the instance resource:

```
ami = data.aws_ssm_parameter.al2023_ami.value
```

The data source is not creating an SSM parameter or an AMI. It reads an AWS-published parameter path.

The lookup is for Amazon Linux 2023 ×86-64. It is Region-dependent and should be checked for availability. Since the parameter points to the latest published image, a later plan can resolve a different AMI. For controlled releases, pin a tested AMI ID or explicitly manage a tested parameter version.

EC2 settings discussed

- Configurable instance type.
- Supplied subnet and security groups.
- Public IPv4 association enabled in the current example.
- Encrypted gp3 root EBS volume.
- Delete root volume on instance termination.
- IMDSv2 enforced with `http_tokens = "required"`.

Outputs

- `instance_id`
- `instance_private_ip`
- `instance_public_ip`

Operational caveats

The root configuration selects the first public subnet with `module.vpc.public_subnet_ids[0]`. That is suitable for a small demonstration but does not provide high availability, load balancing, or fault-tolerant compute. The example enables a public IP but does not open SSH; a public address does not itself grant access. Configure Session Manager prerequisites (instance profile/permissions and connectivity to Systems Manager endpoints or an internet path) before relying on it for administration.

9. CloudWatch monitoring module

Path: `modules/monitoring/`

Purpose

Create an EC2 CPU utilization alarm for a supplied instance.

Inputs

- `alarm_name` — CloudWatch alarm name.
- `instance_id` — EC2 instance identifier.
- `cpu_threshold` — CPU percentage threshold, validated to be greater than 0 and at most 100.
- `tags` — common tags.

Alarm configuration

The example alarm uses:

- Namespace: `AWS/EC2`
- Metric: `CPUUtilization`
- Statistic: `Average`
- Period: 300 seconds (five minutes)
- Evaluation periods: 2
- Comparison: `GreaterThanThreshold`
- Threshold: environment supplied, with examples of 80%, 75%, and 70%.
- Missing-data treatment: `notBreaching`
- Dimension: `InstanceId`

With a 300-second period and two evaluation periods, CloudWatch evaluates two periods. When the metric exceeds the threshold according to the alarm configuration, the alarm can enter ALARM state. This is not a notification by itself.

Outputs

- `cpu_alarm_name`
- `cpu_alarm_arn`

Notification limitation

The module creates an alarm but does not configure an SNS topic, subscription, or notification action in the implementation discussed. To deliver notifications, add an SNS module or inputs for alarm actions and configure/confirm subscriptions.

10. Environment configuration

The project shares the same root module composition and supplies different values through environment-specific `.tfvars` files.

DEV

Setting	Example
Region	`ap-south-1`
Environment	`dev`
VPC name	`enterprise-dev-vpc`
VPC CIDR	`10.10.0.0/16`
Public subnets	`10.10.1.0/24` , `10.10.2.0/24`
Private subnets	`10.10.11.0/24` , `10.10.12.0/24`
Instance type	`t3.micro`
Root volume	8 GiB
CPU threshold	80%

STAGING

Setting	Example
Region	`ap-south-1`
Environment	`staging`
VPC name	`enterprise-staging-vpc`
VPC CIDR	`10.20.0.0/16`
Public subnets	`10.20.1.0/24` , `10.20.2.0/24`
Private subnets	`10.20.11.0/24` , `10.20.12.0/24`
Instance type	`t3.small`
Root volume	15 GiB

Setting	Example
CPU threshold	75%

PROD

Setting	Example
Region	`ap-south-1`
Environment	`prod`
VPC name	`enterprise-prod-vpc`
VPC CIDR	`10.30.0.0/16`
Public subnets	`10.30.1.0/24`, `10.30.2.0/24`
Private subnets	`10.30.11.0/24`, `10.30.12.0/24`
Instance type	`t3.medium`
Root volume	30 GiB
CPU threshold	70%

These are illustrative learning settings, not sizing guidance or an assertion that the project has been deployed. Verify CIDR non-overlap with connected networks and check service quotas and AZ availability.

Important state-isolation distinction

Different `.tfvars` files do not create separate Terraform states. If the same root directory and backend are used, the commands below use the same state location unless an independent backend key/workspace/state arrangement is explicitly configured:

```
terraform plan -var-file="environments/dev/terraform.tfvars"
terraform plan -var-file="environments/staging/terraform.tfva
rs"
terraform plan -var-file="environments/prod/terraform.tfvars"
```

Therefore, separate variable files represent different input configurations, not complete environment isolation. The current project scope excludes remote S3 state, so do not describe these as independently state-isolated environments.

Avoid applying multiple environments against the same local state. A future improvement is to create clearly separated root configurations/state files or implement a remote backend with separate keys and access controls.

11. Common Terraform workflow

Run commands from the project root.

Format

```
terraform fmt -recursive
```

To check without modifying files:

```
terraform fmt -check -recursive
```

Initialize

```
terraform init
```

This downloads required providers and initializes the working directory. Commit `.terraform.lock.hcl` to source control so provider selections and checksums are recorded. Do not commit the `.terraform/` directory.

Validate

```
terraform validate
```

This checks Terraform configuration syntax and internal consistency. It does not prove that every AWS API operation will succeed or that the architecture is secure.

Plan

```
terraform plan -var-file="environments/dev/terraform.tfvars"
```

Review planned resources, modifications, replacements, and deletions. Pay special attention to the plan summary and destructive actions.

Apply and destroy

Deployment and destruction are not part of the confirmed completed work in this documentation. If a later deployment is authorized, apply only a reviewed plan in the intended isolated state context. For a test environment, destroy only after confirming the exact resources and state. Do not use apply or destroy against a shared state accidentally.

12. Troubleshooting notes from implementation

Invalid AWS token

Observed error: STS `InvalidClientTokenId`.

Meaning: The active credential source supplied an invalid, expired, or mismatched token.

Checks:

```
aws configure list
aws sts get-caller-identity
Get-ChildItem Env:AWS*
```

Inspect environment variable names and sources locally; do not paste credential values into chat or documentation. Correct the intended profile, SSO, or temporary-token source before retrying.

List of string required

Observed error: Module expected `list(string)` but received a string.

Cause: CIDR lists must be HCL lists, not single quoted strings.

Correct form:

```
public_subnet_cidrs = [
  "10.10.1.0/24",
```

```
  "10.10.2.0/24"
]
```

Undeclared variable / spelling mismatch

Observed issue: References used misspelled forms such as `avaliability_zones` instead of `availability_zones`.

Resolution: Ensure variable declarations and every reference use the exact same name. Search all Terraform files for the incorrect spelling.

Invalid AWS Region

Observed error: `ap-south-1a` was passed as the provider region.

Cause: An Availability Zone was used where an AWS Region was expected.

Correct values:

- Provider region: `ap-south-1`
- Subnet AZ: account-supported zone names in that Region.

Unsupported module argument

Observed error: Root module passed EC2 arguments not declared by the child module.

Cause: Each argument passed to a module must be declared as an input variable in that module's `variables.tf`. Also verify the module source path and file placement.

Duplicate module declaration

Observed error: Two root module calls were both named `ec2`.

Resolution: Keep a single module call for the current root composition, or use distinct module names when genuinely creating separate module instances in the same root.

13. Security and operational review checklist

Before any AWS deployment:

- [ ] Confirm the AWS account and intended Region with `aws sts get-caller-identity`.
- [ ] Confirm the target account is the intended sandbox/development account.
- [ ] Review the full Terraform plan, including replacements and deletions.
- [ ] Confirm VPC and subnet CIDRs do not overlap existing networks.
- [ ] Verify selected Availability Zones are available in the account.
- [ ] Review ingress and egress rules; avoid unnecessary public exposure.
- [ ] Keep SSH closed to the public internet; prefer Systems Manager Session Manager.
- [ ] Verify EC2 IAM instance profile and SSM connectivity if Session Manager is required.
- [ ] Confirm EBS encryption and IMDSv2 settings.
- [ ] Review the selected AMI and instance type for the Region and workload.
- [ ] Review CloudWatch alarm dimensions, threshold and missing-data behaviour.
- [ ] Add notification actions only with a tested SNS topic/subscription if required.
- [ ] Estimate charges for EC2, EBS, public IPv4, data transfer, CloudWatch, and other enabled services.
- [ ] Protect state files; Terraform state may contain sensitive values.
- [ ] Keep secrets out of Git and variable files.
- [ ] Confirm the active state file belongs to the intended environment before applying.

14. Cost-awareness notes

The project uses a cost-conscious learning approach. Planning does not create billable infrastructure. Actual costs depend on Region, runtime, instance type, storage, public IPv4 usage, traffic, CloudWatch usage, and account-specific free-tier eligibility.

Potential charge areas:

- EC2 instance runtime.

- EBS volumes and snapshots.
- Public IPv4 addresses.
- Internet data transfer.
- CloudWatch alarms, metrics, dashboards, and API usage as applicable.
- NAT Gateway if added later (not part of current design).

No exact cost estimate is included because deployment duration, account eligibility, traffic, and current pricing have not been verified.

15. Recommended future enhancements

These are recommendations, not delivered features:

1. Add validation for subnet/AZ list lengths, CIDR correctness, and CIDR overlap.
2. Improve security-group rule handling so multiple IPv4 CIDRs and IPv6 are modeled safely and explicitly.
3. Add a private EC2 deployment option and remove unconditional public-IP association from the generic EC2 module.
4. Add IAM instance profile and Systems Manager connectivity options for secure administration.
5. Add SNS notification actions for CloudWatch alarms.
6. Introduce independent root/state boundaries for DEV, STAGING, and PROD.
7. Consider S3 remote state with versioning, encryption, restricted IAM, and S3 native lockfiles in a later iteration (currently out of scope).
8. Add automated checks such as `terraform fmt -check`, `terraform validate`, and policy/security scanning in CI.
9. Add module README files documenting inputs, outputs, examples, and compatibility.
10. Add deployment smoke tests and teardown evidence.

16. Portfolio summary

Resume-ready project description

Reusable Enterprise AWS Terraform Modules — Designed a modular Infrastructure as Code project using Terraform to standardize AWS networking, compute, security, and monitoring. Built reusable modules for VPCs, public/private subnets, route tables, security groups, EC2 instances, and CloudWatch CPU alarms. Parameterized infrastructure for DEV, STAGING, and PROD using typed variables, environment-specific `.tfvars`, module outputs, resource tags, and validation. Generated Terraform plans for all three configurations. Remote S3 state and live deployment validation are not included in this iteration.

Interview talking points

- Why use modules rather than copy-paste resource blocks?
- What is the difference between a Terraform resource and data source?
- How does `count` work and why does indexing start at zero?
- How does `merge()` combine common and resource-specific tags?
- How does a child module export IDs to its parent?
- Why are Region and Availability Zone different?
- Why is a public subnet not automatically secure or publicly reachable?
- How does the EC2 module obtain an AMI from SSM Parameter Store?
- What does a CloudWatch alarm evaluate?
- Why do different `.tfvars` files not automatically isolate Terraform state?
- What does `terraform plan` prove, and what does it not prove?

17. Completion definition

This iteration can be considered configuration-development complete when all Terraform files are reviewed, formatting and validation pass, the three plan commands succeed without unexpected destructive changes, and documentation matches the repository.

It should only be described as deployment-tested after a real environment has been applied, AWS resources and connectivity have been verified, CloudWatch

alarm behaviour has been tested, costs/resources have been checked, and cleanup or handover has been recorded.

Current truthful status: Module configurations and three environment plans were reported successful during the build. The full repository has not been independently inspected in this conversation, and live deployment/runtime tests are not confirmed. S3 remote backend remains intentionally out of scope.